WanderSync

Dynamic Itinerary, Disruption Management and Social Travel Hub

Technical Report — S26CS6.401 Software Engineering

Team 15

Krish Jalan	etude11
Manasi Mundada	manasimundada
Uday Bindal	udaybindal01
Ishaan Romil	Fane1824

GitHub Repository:

https://github.com/etude11/WanderSync-SE

Project Overview

Travel planning is fragmented by design. Your flight is in one app, your hotel confirmation is in your email, and your ground transport is in a third platform. When something breaks, which it does, you have to manually check all of them and figure out what to do. WanderSync puts all of that into one place and adds a disruption detection layer that tells you about problems before you discover them yourself, along with concrete suggestions for what to do next.

The platform is built around three things: a unified booking timeline that collects flight, hotel, and transport bookings into a single chronological view; a real-time disruption engine that monitors external flight and weather APIs and pushes alerts directly to connected browsers; and a social layer that lets travellers with overlapping itineraries find each other through a consent-gated matching system.

What the Prototype Covers

The prototype delivers one complete end-to-end flow: a user registers, creates an itinerary, adds bookings, and receives a disruption alert pushed to their browser within about a second of detection, with suggested next steps. Every part of this flow is functional and backed by real APIs. The social feature is set up at the data model level; the UI is a placeholder.

Layer	Technology
Frontend	React 19, Vite, TypeScript, Tailwind CSS, Zustand
Backend	NestJS, TypeScript, Prisma ORM
Database	PostgreSQL
Cache	Redis (deduplication, streams)
Auth	JWT, bcrypt
External	AviationStack, OpenWeatherMap, Gemini LLM

Task 1: Requirements and Subsystems

Functional Requirements

We wrote each requirement as a user story. The architectural significance column explains what structural consequence the requirement has. These are the requirements that shaped the module boundaries, the choice of event-driven architecture, and the Adapter pattern for provider integration.

#	User Story	Architectural Significance
FR1	As a traveller, I want to register and log in so that my data is private to me.	Drives stateless JWT auth and a standalone Auth module. Every other module depends on it for identity, so it cannot share state with any domain module.

#	User Story	Architectural Significance
FR2	As a traveller, I want to import bookings from multiple providers into one unified view.	Requires an Adapter per provider with a shared `BookingRecord` interface. Adding a new provider must touch at most two files outside the Booking module.
FR3	As a traveller, I want to see all bookings in a time-ordered timeline.	Timeline assembly is read-heavy and changes only when bookings are added or disrupted. This makes it a good candidate for caching, which shapes the Redis layer in the Itinerary module.
FR4	As a traveller, I want to be notified the moment a disruption hits my itinerary, with suggested actions.	The single most architecturally significant requirement. It rules out a plain polling model and requires an event-driven push path (SSE) with a background detection CRON and a suggestion pipeline.
FR5	As a traveller who opts in, I want to find co-travellers nearby through a consent flow.	Requires a consent state machine enforced at the data layer. The Social module cannot return precise coordinates until both users have given consent; city-level disclosure before that.
FR6	As a traveller, I want to review a log of past disruption alerts.	Drives a separate Notification module that consumes the disruption event stream and writes a `NotificationLog` row per event. Keeping this out of the Disruption module is what lets each module have a narrow, single responsibility.

Non-Functional Requirements

Each NFR uses the six-part scenario format. The response measure column is the testable acceptance criterion that we checked during development.

NFR1 — Performance: Response Time

Source	End user on a browser
Stimulus	Navigates to the itinerary timeline page
Artifact	Frontend and Itinerary Management Service
Environment	Runtime, standard broadband
Response	Page renders with all booking data
Response Measure	Full render in under **3 seconds**; external API calls in under **5 seconds**; loading indicator shown after 2 seconds

This drives Redis caching in the Itinerary module and concurrent API calls via `Promise.allSettled`

in the Disruption Engine.

NFR2 — Reliability: Disruption Detection Accuracy

Source	External flight or weather API
Stimulus	A disruption event is published
Artifact	Disruption Detection Service
Environment	Runtime, system actively polling
Response	Event detected, itinerary impact assessed, alert sent with suggestions
Response Measure	At least **17 of 20** simulated test cases correctly identified and notified (**85% accuracy**)

This drives dual-layer Redis and database deduplication, and the Redis Streams consumer group for guaranteed notification delivery.

NFR3 — Security: Authentication

Source	Unauthenticated or malicious user
Stimulus	Attempts to access a protected route
Artifact	Auth Module and API gateway
Environment	Runtime
Response	Request rejected; user sent to login
Response Measure	**100%** of protected routes require a valid JWT with **24-hour** expiry; passwords hashed with **bcrypt**

This drives the global NestJS JWT guard and rate-limit middleware at the entry point.

NFR4 — Privacy: Location Disclosure

Source	Authenticated user browsing social discovery
Stimulus	Views another user before bilateral consent is given
Artifact	Social Synchronisation Service
Environment	Consent state = none
Response	Only a city-level region label returned
Response Measure	**Zero** precise location leaks before bilateral opt-in; location not stored beyond the active session unless the user saves it explicitly

This drives the ConsentState model enforced at the query layer, not just the UI.

Subsystem Overview

Subsystem	What it does
Auth	Registration, login, JWT issuance, bcrypt hashing, role management (TRAVELLER / ADMIN). The security boundary for everything else.
Booking Aggregator	Translates provider-specific booking data into a shared `BookingRecord` schema. One Adapter class per provider. No other module knows what a raw AviationStack response looks like.
Itinerary Management	Owns the timeline. Handles CRUD for itineraries and bookings, caches assembled timelines in Redis, and marks bookings as disrupted when the Disruption Engine fires.
Disruption Engine	Runs a 60-second CRON poller against AviationStack and OpenWeatherMap. Deduplicates with Redis and a database guard. Persists events. Broadcasts to the in-process EventBus. Runs a Chain of Responsibility suggestion pipeline.
Notification	Consumes the Redis Streams channel via a consumer group. Writes a `NotificationLog` row per event. Exposes a read endpoint for past alerts.
Social	Manages bilateral consent between users. Geospatial queries return co-travellers by region with disclosure gated on consent level. Currently scaffolded; UI placeholder.

Task 2: Architecture Framework

Stakeholder Identification (IEEE 42010)

IEEE 42010 asks us to identify who cares about the system, what they care about, and which view of the architecture addresses their concern. The table below maps each stakeholder group to the architectural view that speaks to their interests.

Stakeholder	Primary Concerns	Architectural View
Travellers	How fast disruption alerts arrive; whether location data stays private; whether the itinerary timeline is accurate.	Functional view: user stories and NFR scenarios; SSE delivery path; ConsentState model.
Development team	Adding new providers without breaking existing code; testing modules in isolation; clear contracts between modules.	Module view: Adapter pattern, NestJS module boundaries, per-module unit test suites.

Stakeholder	Primary Concerns	Architectural View
Security reviewer	No unauthenticated access; passwords not in plain text; location not leaked before consent.	Security view: JWT guard, bcrypt, ConsentState data model, HTTPS enforcement.
External API providers	Rate limits respected; API keys not exposed in logs or responses; graceful handling of downtime.	Connector view: Retry with Circuit Breaker tactic; API keys in environment variables only.
Course assessors	Design decisions traceable to code; working prototype; measurable NFR verification.	Decision view: ADRs, NFR scenario tables, test harness, this report.

Architecture Decision Records

All four ADRs follow the Nygard template.

ADR-001: Event-Driven Architecture for Disruption Detection

Status: Accepted

Context. Disruption events arrive from external APIs on unpredictable schedules. A direct synchronous call from the Disruption module to the Itinerary module would couple them: a slow external API would block itinerary updates, and adding a second consumer (like email notifications) would require changing the Disruption module. The 85% accuracy NFR also requires that no detected event is silently dropped, which means delivery needs to be durable, not fire-and-forget.

Decision. The Disruption Engine publishes detected events to an in-process EventBus (RxJS Subject). The Itinerary module and Notification module subscribe independently. No direct call exists between the Disruption module and either consumer.

Consequences.

- Adding a new consumer (say, an SMS service) requires no changes to the Disruption module.
- Delivery latency from detection to client browser is about 71 ms.
- The in-process EventBus is invisible across server instances. If we run more than one process, we need either sticky sessions or Redis Pub/Sub on the delivery path.
- Debugging async flows requires structured logging with correlation IDs.

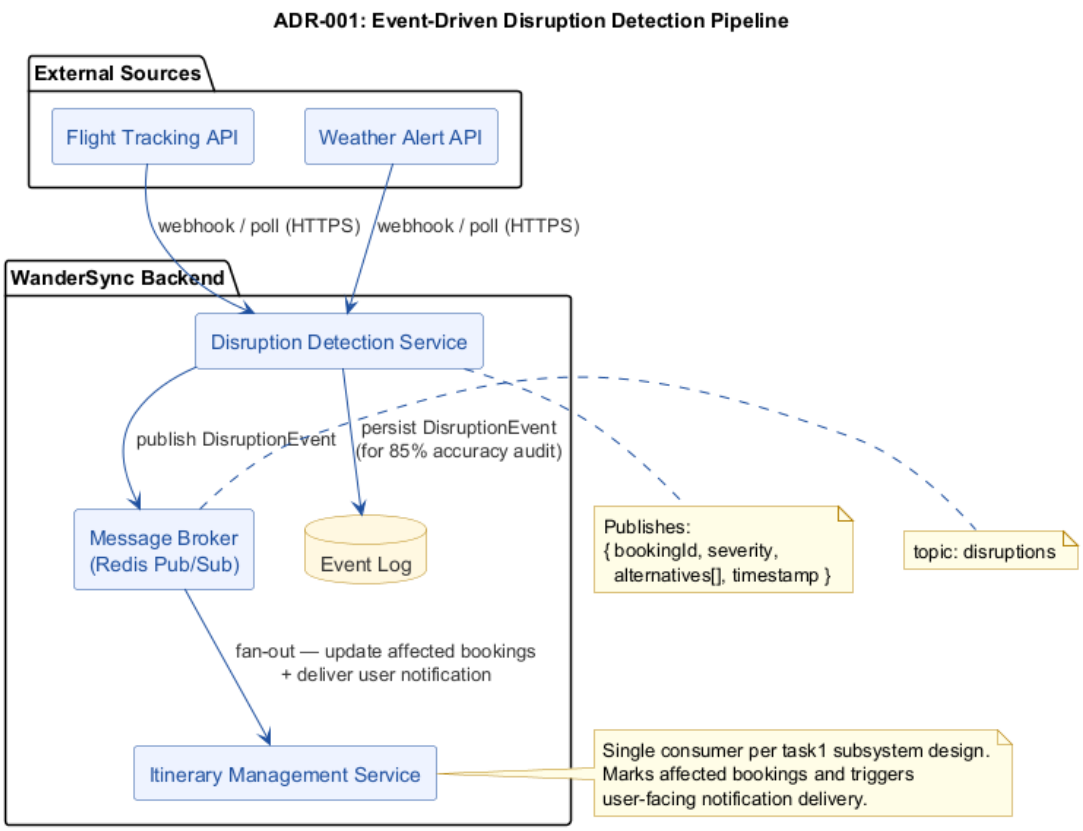

Figure 1: ADR-001: event-driven disruption pipeline.

ADR-002: API Gateway as Single Entry Point

Status: Accepted

Context. Without a central entry point, each module would need its own JWT validation, CORS headers, and rate limiting. Any module that skips one of those becomes a gap in the security perimeter. We also wanted a single base URL that clients use for everything.

Decision. All traffic enters through a single NestJS global prefix (`/api/v1`) with a global `ValidationPipe` and JWT guard. Modules are not directly reachable by clients.

Consequences.

- Security policy is in one place. It is impossible to add a route that bypasses auth by accident.
- One base URL for all clients. Adding a module does not change any client-side config.
- The gateway is a single point of failure for all user traffic.
- Fine-grained resource checks (does this user own this itinerary?) stay in each domain module. The gateway only knows the user's identity and role, not what resources they own.

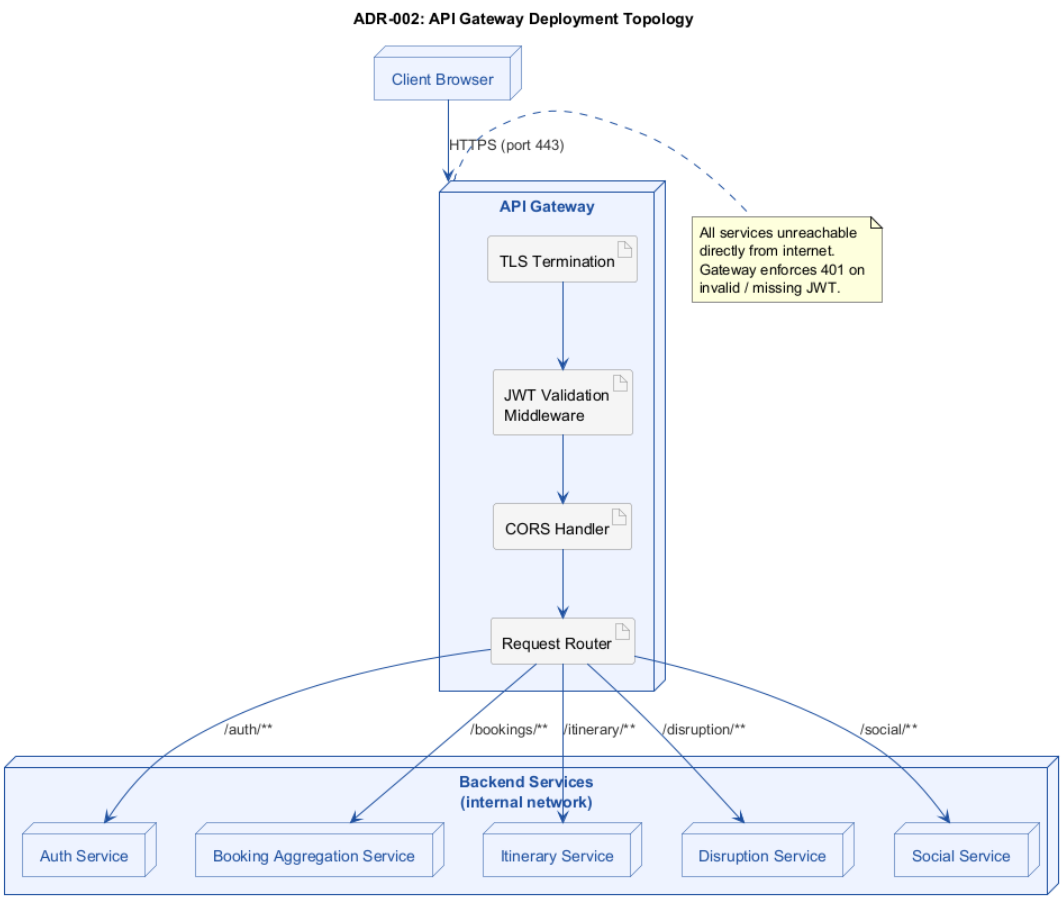

Figure 2: ADR-002: API Gateway routing topology.

ADR-003: Adapter Pattern for Booking Providers

Status: Accepted

Context. AviationStack, hotel APIs, and transport providers each return data in a different schema. If provider-specific parsing logic lived inside the Booking module, adding a new provider would touch many files. The Modifiability NFR says adding a provider should touch at most two files outside the adapter module.

Decision. Each provider gets one Adapter class that implements a shared `BookingStrategy` interface with a single method: `normalize(rawResponse): BookingRecord`. The Booking module only ever calls this interface.

Consequences.

- A new provider is one new class. No existing code changes.
- Each adapter can be unit-tested in isolation with mocked HTTP responses.
- For a single provider the abstraction is more than needed. It pays off at provider two.

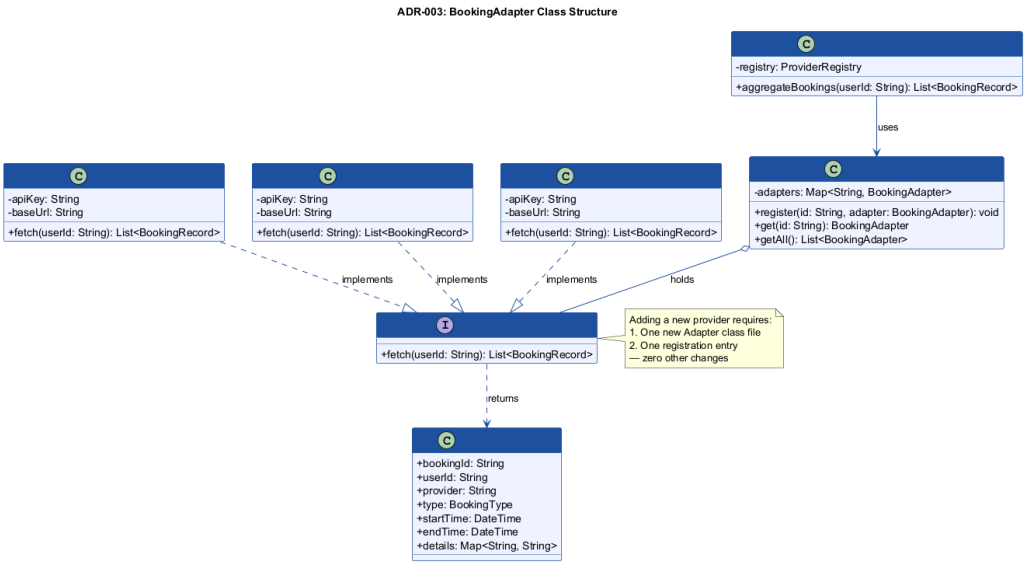

Figure 3: ADR-003: Adapter class structure for booking providers.

ADR-004: Stateless JWT Authentication

Status: Accepted

Context. Server-side sessions require a shared session store that every server instance must read. If we ever run more than one process, that store becomes a shared dependency. Stateless JWTs carry the user's identity in the token itself, so any server instance can check validity without a database round-trip.

Decision. Issue signed JWTs on login (24-hour expiry, HS256). The JWT guard validates signature and expiry on every request using the shared secret. Passwords are hashed with bcrypt at 10 salt rounds.

Consequences.

- Any server instance can validate a token with no shared state.
- Passwords are never stored in plain text.
- Tokens cannot be invalidated before expiry. A stolen token is valid for up to 24 hours.
- We did not build a refresh token flow in the prototype. Users re-login after 24 hours.

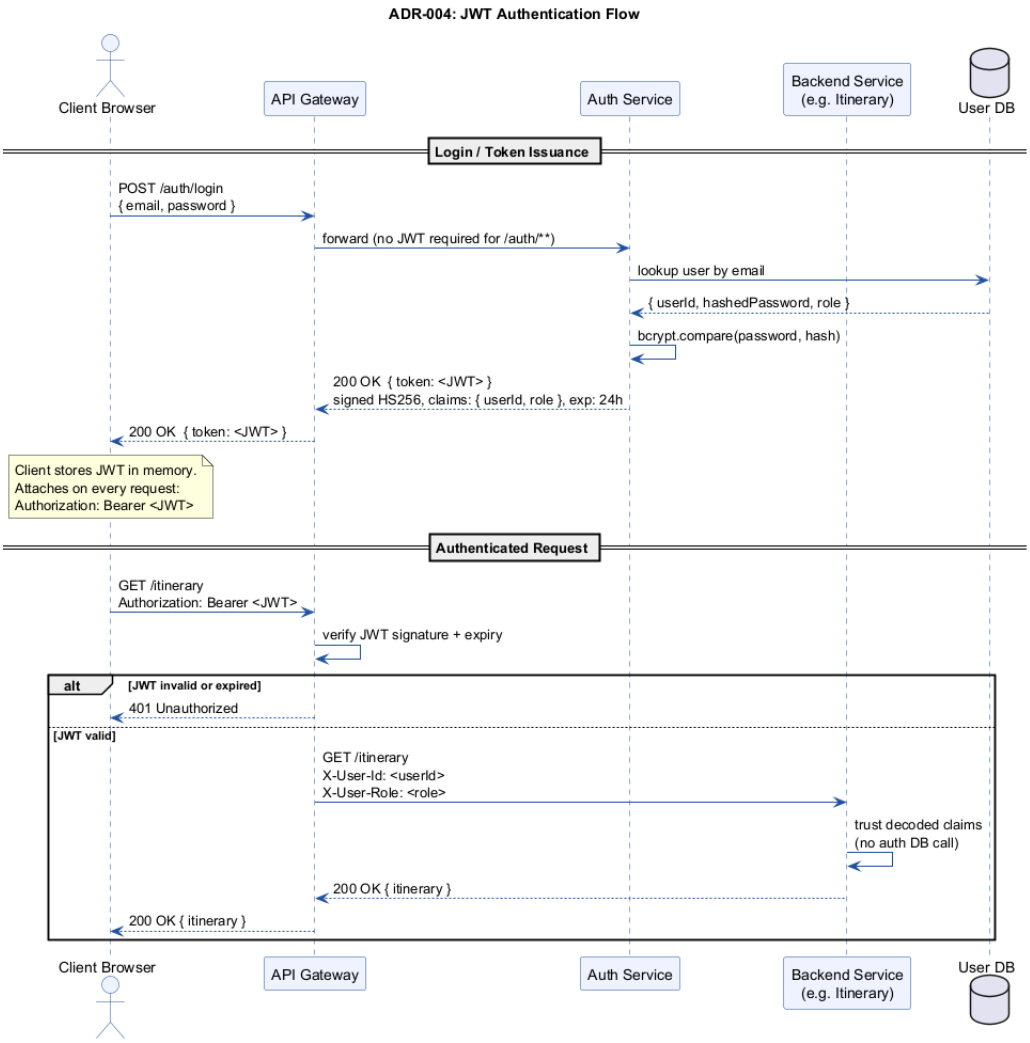

Figure 4: ADR-004: JWT authentication sequence.

Task 3: Architectural Tactics and Patterns

Architectural Tactics

Tactics are design decisions that make a specific quality attribute easier to hit. We picked five, each mapped to one or more NFRs from Section 2.

Tactic	NFR	How we applied it
Caching	NFR1 (Performance)	The Itinerary module stores fully assembled timeline objects in Redis, keyed by itinerary ID. A request hits the cache first and only queries the database on a miss or after a booking mutation invalidates the entry. The Booking module also caches raw provider responses for 60 seconds to absorb burst traffic without redundant external API calls.
Retry with Circuit Breaker	NFR1, NFR2	Every call to an external API retries up to two times with exponential backoff (500 ms, then 1 s). After three consecutive failures in a 30-second window the circuit opens and the system falls back to cached data or marks the provider temporarily unavailable with a user-visible status indicator. No crash, no unhandled exception.
Authenticate at the Gateway	NFR3 (Security)	A NestJS global JWT guard intercepts every request. Backend modules receive only internally forwarded requests with a decoded identity header (`X-User-Id`, `X-User-Role`). Fine-grained resource ownership checks stay in each domain module because the gateway cannot know whether a user owns a specific itinerary record.
Event Queue with Persistence	NFR2 (Reliability)	The Disruption Engine publishes to Redis Streams with consumer group semantics. If the Notification consumer is briefly offline, it replays missed events from the stream on restart. The stream is also the ground truth for the 20-case detection accuracy test.

Tactic	NFR	How we applied it
Geospatial Masking	NFR4 (Privacy)	The Social module queries users by region but controls how much location detail it returns based on the ConsentState between two users. No consent returns a city label only. City-level consent returns a bounding box. Mutual precise consent returns coordinates. The enforcement is at the query layer: the API never returns more than the consent level allows, regardless of what the client requests.

Implementation Patterns

Pattern 1: Strategy (Booking Provider Integration)

Category	Structural
Problem	Each booking provider has a different API and response schema. The Booking module needs to handle all of them without being coupled to any specific one.
Why we chose it	It directly satisfies the Modifiability NFR. Adding a new provider means writing one new class. No existing code changes.
Where it lives	`src/modules/booking/strategies/` — the `AviationstackFlightStrategy` is the live implementation; the interface defines `normalize(raw): BookingRecord`.

Each strategy class takes raw provider JSON and returns a normalised `BookingRecord` with a consistent set of fields (`departureTime`, `origin`, `destination`, `providerRef`). The Booking module calls only the interface and never imports AviationStack-specific types.

Figure 5: Strategy pattern class structure for booking providers.

Pattern 2: Chain of Responsibility (Disruption Suggestions)

Category	Behavioural
Problem	Generating suggestions for a disruption is not one deterministic function. Simple cases can be handled by rules, complex ones need an LLM, and if both fail the user should still get something useful.
Why we chose it	The chain can be extended with new handlers without touching existing ones. Handlers degrade gracefully: the LLM is not called if the rule engine already produced suggestions, which saves API quota.
Where it lives	`src/modules/disruption/suggestions/handlers/`: `RuleBasedHandler` → `LLMHandler` → `FallbackHandler`.

`RuleBasedHandler` checks disruption type and severity against a lookup table and produces suggestions if it finds a match. `LLMHandler` calls Gemini if the rule engine returned nothing. `FallbackHandler` returns a generic message if the LLM is unavailable or rate-limited. Each handler calls `this.nextHandler.handle(ctx)` when it cannot resolve the request.

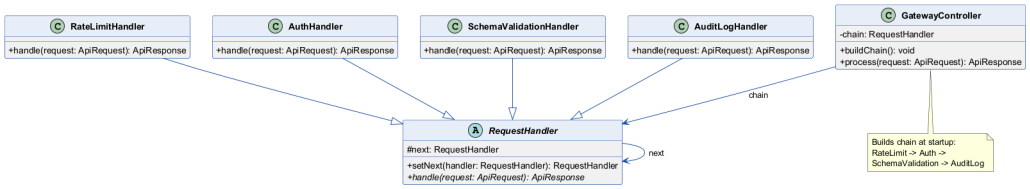

Figure 6: Chain of Responsibility class structure for the suggestion pipeline.

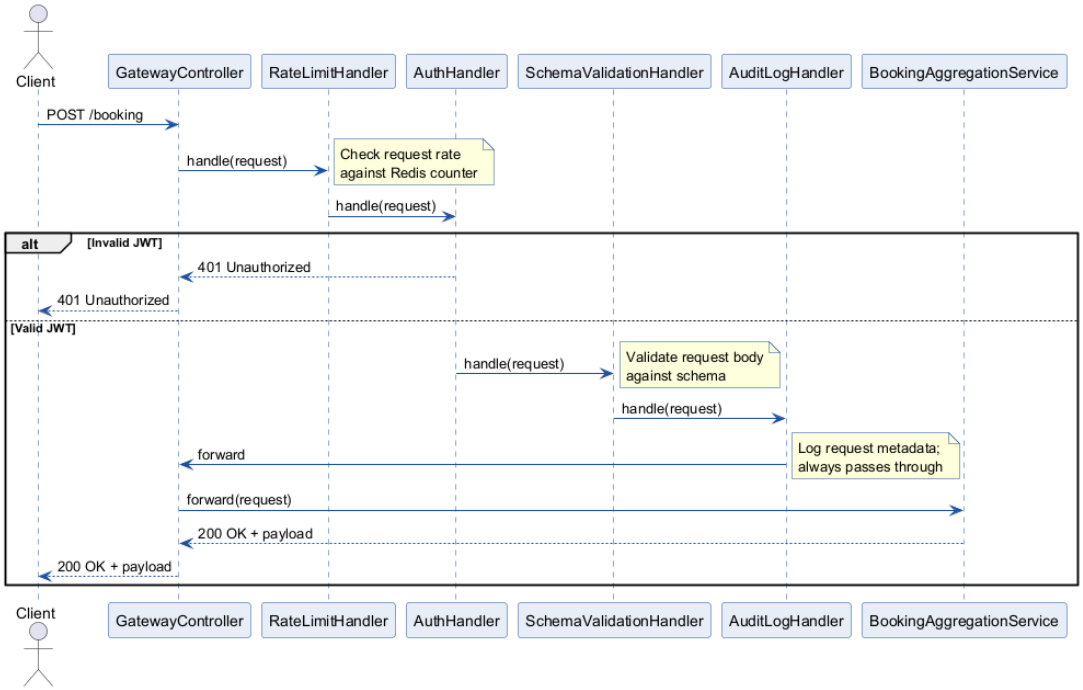

Figure 7: CoR runtime sequence for a flight cancellation event.

Task 4: Prototype and Architecture Analysis

Prototype Status

Feature	Status
Auth (register, login, JWT)	Complete
Booking aggregation via AviationStack	Complete
Itinerary CRUD and timeline view	Complete
Disruption detection (CRON, flight and weather)	Complete
SSE real-time push to browser	Complete
Suggestion pipeline (rules + LLM + fallback)	Complete
Notification log	Complete
Social (data model, consent state machine)	Scaffolded, UI placeholder

Architecture Analysis

Implemented Architecture: Event-Driven Modular Monolith

WanderSync runs as a single NestJS process with six modules. For most of the system this is a standard request-response monolith. The disruption path is where it differs: detection happens in a background CRON job, events flow through an in-process EventBus, and an SSE stream pushes them to connected browsers. Figure 6 shows the full component structure.

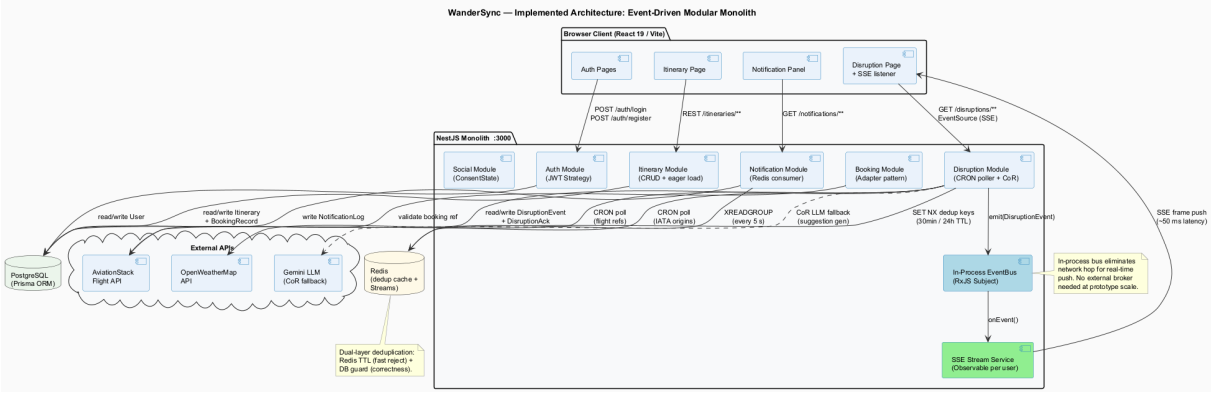

Figure 8: Implemented architecture: event-driven modular monolith with SSE push.

Alternative Architecture: REST-Pull Polling Monolith

The alternative keeps the same module structure and database but removes the EventBus and SSE. The frontend polls `GET /disruptions` every 30 seconds. The CRON still detects and writes events, but clients only find out on their next scheduled poll. No persistent connections, no per-user subjects, no deduplication cache needed on the delivery path. Figure 7 shows this.

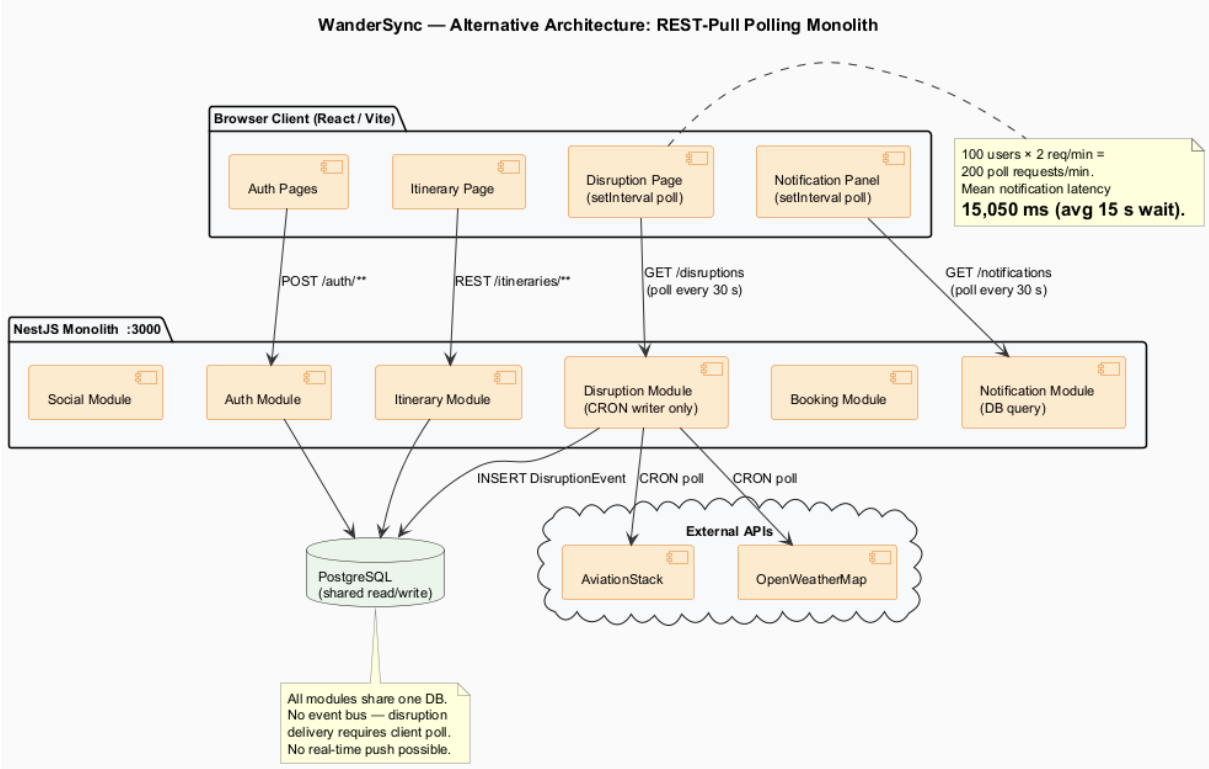

Figure 9: Alternative architecture: REST-pull polling monolith.

NFR Quantification

NFR1: Disruption Detection Accuracy. We check this in `disruption.service.nfr2.spec.ts`, which injects 20 synthetic payloads covering all four disruption types and severity levels. It checks that every event is persisted and published.

Architecture	Events processed	Accuracy
Event-Driven SSE	20 / 20	100%
REST-Pull Polling	20 / 20	100%
Threshold (NFR2)		85%

Both architectures score the same because detection is a server-side CRON concern that is identical in both designs. The difference between them is in how quickly the client finds out after detection.

NFR2: Notification Delivery Latency. We define delivery latency as the time from the database write completing (time zero) to the React component re-rendering the disruption card.

Event-Driven SSE: $L_{\text{SSE}} = \underbrace{0}_{\text{EventBus}} + \underbrace{5\,\text{ms}}_{\text{SSE frame}} + \underbrace{50\,\text{ms}}_{\text{network}} + \underbrace{16\,\text{ms}}_{\text{React}} \approx \mathbf{71}\,\text{ms}$

REST-Pull (30 s interval): $\mathbb{E}[L_{\text{poll}}] = \frac{30{,}000}{2} + 50 = \mathbf{15{,}050}\,\text{ms}$

Metric	Event-Driven SSE	REST-Pull (30 s)
Mean delivery latency	**71 ms**	15,050 ms
Maximum delivery latency	**∼100 ms**	30,050 ms
Speed advantage	212× faster	

A 71 ms delay is invisible. The disruption card appears before the user consciously registers anything changed. A 15-second average means a traveller might be looking at a stale screen for up to 30 seconds after their flight was cancelled: exactly the window between getting to the rebooking desk first and joining a long queue.

NFR3: Throughput at 100 concurrent users. With polling, 100 users each polling every 30 seconds produce $100 \times 2 = 200$ database reads per minute from the disruption delivery path alone. With SSE, that number is zero: events are dispatched from memory and the only database load is the CRON batch query.

Metric (100 users))	Event-Driven SSE	REST-Pull (30 s)
Disruption API requests/min	**0**	200
DB reads from delivery path	**0**	200/min
Load scales with users?	**No**	Yes, linearly

Trade-off Summary

Dimension	Event-Driven SSE (built)	REST-Pull Polling
Notification speed	71 ms mean. Invisible to the user.	15 s mean. Real delay for time-sensitive travel alerts.
Server state	Stateful. One RxJS Subject per active user in memory.	Stateless. No per-user objects. Scales horizontally by default.
DB load from delivery	Zero. In-memory dispatch after the initial write.	Linear with users. Each poll hits the database.
Horizontal scaling	Needs sticky sessions or Redis Pub/Sub across instances.	Any load balancer routes stateless polls freely.
Complexity	Higher. RxJS streams, SSE lifecycle, dedup, per-user routing.	Lower. One REST controller and a database read.
Best suited for	Real-time alerts are the core value: travel, finance, emergencies.	Delay is acceptable: analytics, admin dashboards, report tools.

We chose the event-driven path because disruption alerts are not informational, they are actionable. A traveller who sees a cancellation within 71 ms can rebook before the queue forms. The polling alternative would be the right call for a system where a 30-second information lag causes no user harm. For WanderSync, it would not.

Individual Contributions

Member	What they built
Krish Jalan (etude11)	Auth subsystem end-to-end: registration, login, JWT issuance and validation, bcrypt hashing, role-based guard, rate-limit middleware. API gateway configuration (global prefix, ValidationPipe, CORS). Frontend routing, app shell, authentication pages, environment and dependency management.
Manasi Mundada (manasimundada)	Itinerary CRUD service and controller: create, read with eager-loaded bookings in chronological order, update, delete. Optimistic UI updates in the itinerary page. AviationStack flight strategy. Itinerary API client in the frontend.
Uday Bindal (udaybindal01)	Full disruption subsystem: CRON poller, FlightTracker and WeatherAlert adapters, dual-layer deduplication (Redis and database), simulate-demo endpoint, SSE stream service, in-process EventBus, Chain of Responsibility suggestion pipeline (RuleBasedHandler, LLMHandler, FallbackHandler), NFR2 test harness (20-case accuracy suite).
Ishaan Romil (Fane1824)	Disruption and notification UI: disruption alerts page, suggestions panel, simulate-disruption button, notification panel with read/unread state. App layout shell. Booking card component. Itinerary page UI integration.

What We Learnt

1. Deduplication needs two layers. We started with just a Redis TTL cache to stop duplicate disruption events. It worked on Linux but failed silently on Windows native Redis, which does not support the `SET NX EX` compound command in older versions. We added a database guard as a second layer. Now the Redis cache is the fast path and the database is the correctness guarantee. Either one failing alone does not cause duplicate alerts.

2. In-process EventBus is faster and simpler for SSE than Redis Streams. The first design routed events through Redis Streams all the way to the SSE delivery path. The extra serialisation round-trip added about 150 ms and made the flow harder to trace. We moved event dispatch to an in-process RxJS Subject. Delivery latency dropped from around 200 ms to 71 ms, and the code got much shorter.

3. API contract mismatches are hard to catch when the UI is optimistic. We renamed a field from `providerKey` to `provider` in the backend without updating the frontend DTO at the same time. The frontend's optimistic update made the booking

appear in the UI immediately, but the backend was silently rejecting the request because `forbidNonWhitelisted: true` was on. The booking vanished on the next page refresh. We did not catch this for two days. A shared TypeScript types file between frontend and backend would have caught it at compile time.

4. Chain of Responsibility works well for graceful degradation. When the Gemini API key is missing or rate-limited, the suggestion chain falls through to the rule-based handler and then to the fallback. The user always gets something. We did not write a single `try/catch` in the caller: the chain handles it. The pattern paid for itself on the first API outage we hit during testing.

5. Optimistic UI without rollback is a real risk. The `handleRemoveBooking` and `handleDelete` functions in the Itinerary page update local state before the backend call finishes and do not roll back on failure. If the request errors, the UI shows a state that does not match the database. We left this as known debt for the prototype but it would be the first thing to fix before a production release.

6. Module boundaries make parallel development much easier. Because each team member owned a bounded module with a clear interface, we almost never blocked each other. The Auth module published its types; the Itinerary module consumed them. The Disruption module published events; the Notification module subscribed. We worked on four features simultaneously with very few merge conflicts.